PROJEKTZUSAMMENFASSUNG

Geofencing LoRaWAN

Überwachung eines Patienten in einer Kontrollzone, mit Alarmen auf dem Bildschirm, per Ton und per SMS

Plattform: The Things Industries (Cluster eu1) · Karte: Leaflet · SMS: Vonage
Zusammenfassung zur Vorbereitung einer Präsentation · aktualisiert am 3. November 2026 · deutsche Fassung
Projektstatus: funktionsfähige Demonstration ohne Hardware (simulierte Positionen und Nachrichten)

1. Das Projekt in Kürze

Ziel. Die Position eines mit einem LoRaWAN-GPS-Gerät ausgestatteten Patienten in Echtzeit überwachen und einen Angehörigen oder eine Pflegekraft sofort benachrichtigen, wenn der Patient seine Kontrollzone verlässt, und danach, wenn er zurückkehrt.

Besonderheit. Für die Anzeige muss kein Anwendungsserver betrieben werden: Eine einfache Webseite fragt direkt die Plattform The Things Stack ab. Für die SMS kommt ein einziges kleines lokales Programm hinzu, damit die Zugangsdaten des SMS-Anbieters nie offengelegt werden.

Nachrichten. Der Angehörige kann auch mit dem Patienten sprechen (Text, in Sprache umgewandelt, oder Sprachnachricht); der Patient antwortet mit Sprachnachrichten. Das Gerät hat weder Tastatur noch Bildschirm (Abschnitte 10 und 11).

Ausgangsbeschränkung. Keine Hardware verfügbar (weder Gerät noch GPS noch Gateway). Die Positionen werden daher mit dem Werkzeug „Simulate uplink" von The Things Stack und zwei eigens für das Projekt geschriebenen Seiten (Simulator und simuliertes Gerät) simuliert; die LED des Geräts wird durch eine virtuelle LED ersetzt. Der Code des echten Geräts ist für den Tag geschrieben, an dem die Hardware verfügbar ist.

35	≈ 6 000	7	3+
Projektdateien	Zeilen Code und Konfiguration	Anleitungen und README	Punkte für die Zone

Was das System heute kann

- Die aus The Things Stack dekodierten Positionen live empfangen (HTTP-Ereignisstrom).
- Den Sensor, seine Strecke und seine Adresse auf einer Karte anzeigen.
- Eine **Polygonzone** überwachen, definiert durch 3 oder mehr GPS-Punkte (Dreieck, Viereck, Fünfeck …), oder aus einer **Adresse mit Radius** gezeichnet.
- Austritt und Rückkehr erkennen, mit einer Toleranz von 5 m gegen Fehlalarme.
- Alarmieren: blinkendes Banner, Pieptöne, virtuelle LED, „LED"-Befehl an das Gerät (Downlink).
- **Sich mit einem einzigen Befehl installieren** lassen, auf jedem Rechner (Windows, macOS, Linux), auch ohne Entwickler zu sein.
- Eine **SMS** „Ihr Patient hat seine Kontrollzone gerade verlassen" und danach „Ihr Patient ist gerade in seine Kontrollzone zurückgekehrt" senden, an Nummern, die direkt in der Seite verwaltet werden.
- **Nachrichten austauschen** mit dem Patienten und das Gerät in einer eigenen Seite simulieren (Karte, Mikrofon, Nachrichten).

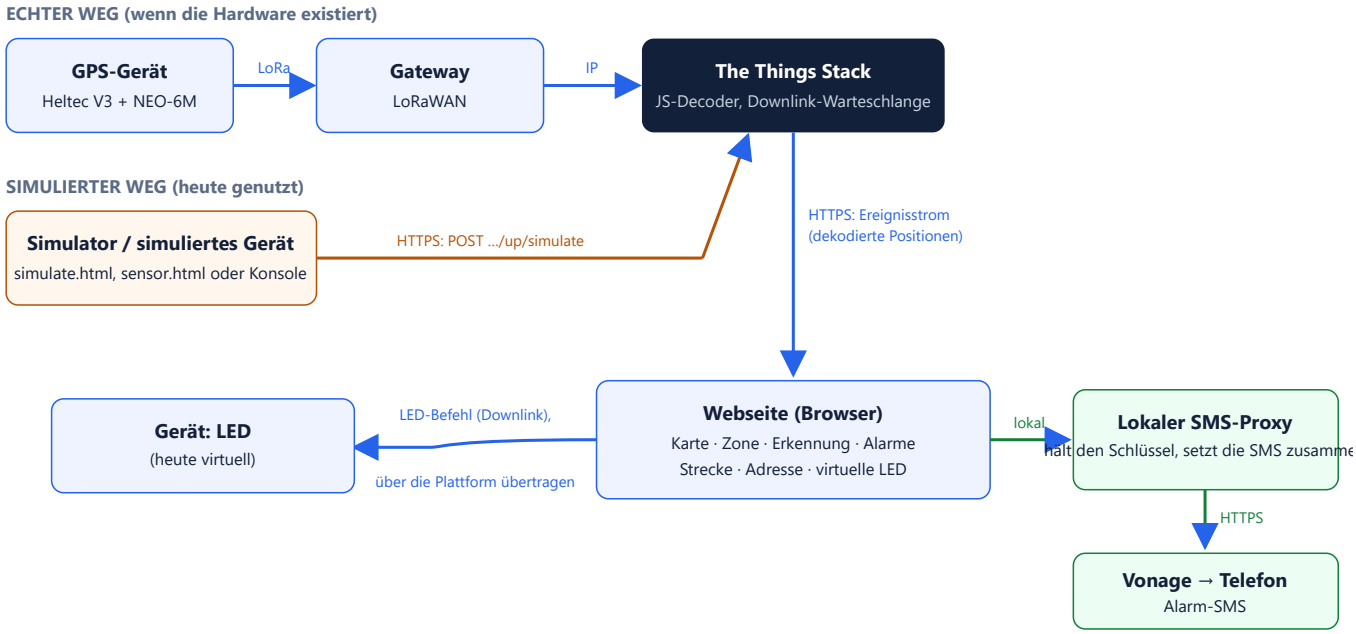
Wo stehen wir?

Stufe	Inhalt
Validiert	Konfiguration von The Things Stack, Dekodierung der Positionen, Empfang in der Seite, Test-SMS auf einem Telefon empfangen.
In Simulation getestet	Polygonzone, Erkennung, Strecke, Alarme, LED, SMS-Nachrichten, Schutzmaßnahmen, Zustellverfolgung, Simulator, Nachrichten und simuliertes Gerät (74 Tests des Relays).
Nicht getestet	Code des Geräts auf einer echten Platine (Position, Stimme), realer Empfang eines Downlinks, Ablauf „Uplink → Alarm → SMS" unter echten Bedingungen.

Die Einzelheiten zu jedem Element stehen in Abschnitt 14 „Stand der Arbeiten".

2. Architektur und Datenfluss

Zwei Wege speisen die Seite: der **echte** Weg (mit dem Gerät, wenn es existiert) und der **simulierte** Weg (heute genutzt). Ab The Things Stack ist alles identisch. Dieses Schema zeigt die Positionen und die LED; die Nachrichten haben ihr eigenes Schema in Abschnitt 10.



Aufgabe jeder Komponente

Komponente	Aufgabe
GPS-Gerät	Misst die Position und sendet sie per LoRaWAN alle 30 s (8 Byte); schaltet auf Befehl seine LED ein oder aus.
The Things Stack	LoRaWAN-Netz in der Cloud: empfängt, entschlüsselt und dekodiert den Rahmen (JavaScript-Formatter), reiht die Befehle in eine Warteschlange ein.
Webseite	Empfängt die Positionen, berechnet die Lage zur Zone, löst die Alarme aus und sendet den Downlink.
Lokaler SMS-Proxy	Einziger Inhaber des Schlüssels des SMS-Anbieters und der Nummern; setzt den Text zusammen; begrenzt die Zahl der Sendungen.
Simulator	Ersetzt das Gerät: speist Positionen „in der Zone“, „außerhalb der Zone“ oder eine automatische Strecke ein.
Seite des simulierten Geräts	Simuliert das ganze Gerät: Karte, Position, Mikrofon und Nachrichten. Sendet echte Uplinks an die Plattform (Abschnitt 11).
Lokales Sprach-Relay	Kleiner, in den lokalen Server eingebundener Dienst: speichert die Sprachnachrichten und Texte (7 Tage), erzeugt die Sprachsynthese des Textes und liefert das Audio per WLAN an das Gerät (Abschnitt 10).

> **Warum ein HTTP-Strom und nicht MQTT?** Der Browser kann nur WebSocket sprechen. Die auf dem Cluster getesteten MQTT-Ports (8883 und 443) bieten kein MQTT-WebSocket. Die Plattform bietet dagegen einen HTTP-Ereignisstrom, der direkt aus der Seite nutzbar ist.

3. Der Funkrahmen von 8 Byte

Jede Position passt in 8 Byte auf Port 1: zuerst der Breitengrad, dann der Längengrad, jeweils mit 10^7 multipliziert und als vorzeichenbehaftete 32-Bit-Ganzzahl (Big-Endian) kodiert. Die Genauigkeit beträgt 10^{-7} Grad, also etwa 1 cm.



Decoder (Uplink)

`ttn/uplink_decoder.js` : akzeptiert Port 1 (Position) und Port 3 (Lesebestätigung); bei der Position prüft er die Länge (8 Byte), baut die beiden vorzeichenbehafteten Ganzzahlen wieder auf (negative Werte werden behandelt) und lehnt Koordinaten außerhalb der Grenzen ab.

Beispiel: `1D1EECF00166EAD0` → `{ "lat": 48.8566, "lng": 2.3522 }`

Encoder (Downlink)

`ttn/downlink_encoder.js` : wandelt `{ "alert": true }` in das Byte `0x01` (LED blinkt) und `{ "alert": false }` in `0x00` (LED aus) auf Port 2 um; einen Text in Bytes auf Port 4; das Sprachsignal auf Port 5.

Die Dekodierung wurde in der Konsole der Plattform geprüft.

Die verwendeten LoRaWAN-Ports

Port	Richtung	Inhalt
1	Gerät → Seite	Position: 8 Byte (oben)
2	Seite → Gerät	LED-Befehl: 0x01 blinkt, 0x00 aus
3	Gerät → Seite	Lesebestätigung eines Textes: 1 Byte, die Nummer der Nachricht
4	Seite → Gerät	Textnachricht: 1 Byte Nummer + UTF-8-Text (höchstens 49 Byte)
5	Seite → Gerät	Signal „eine Sprachnachricht wartet“: 1 Byte (das Audio läuft über das WLAN)

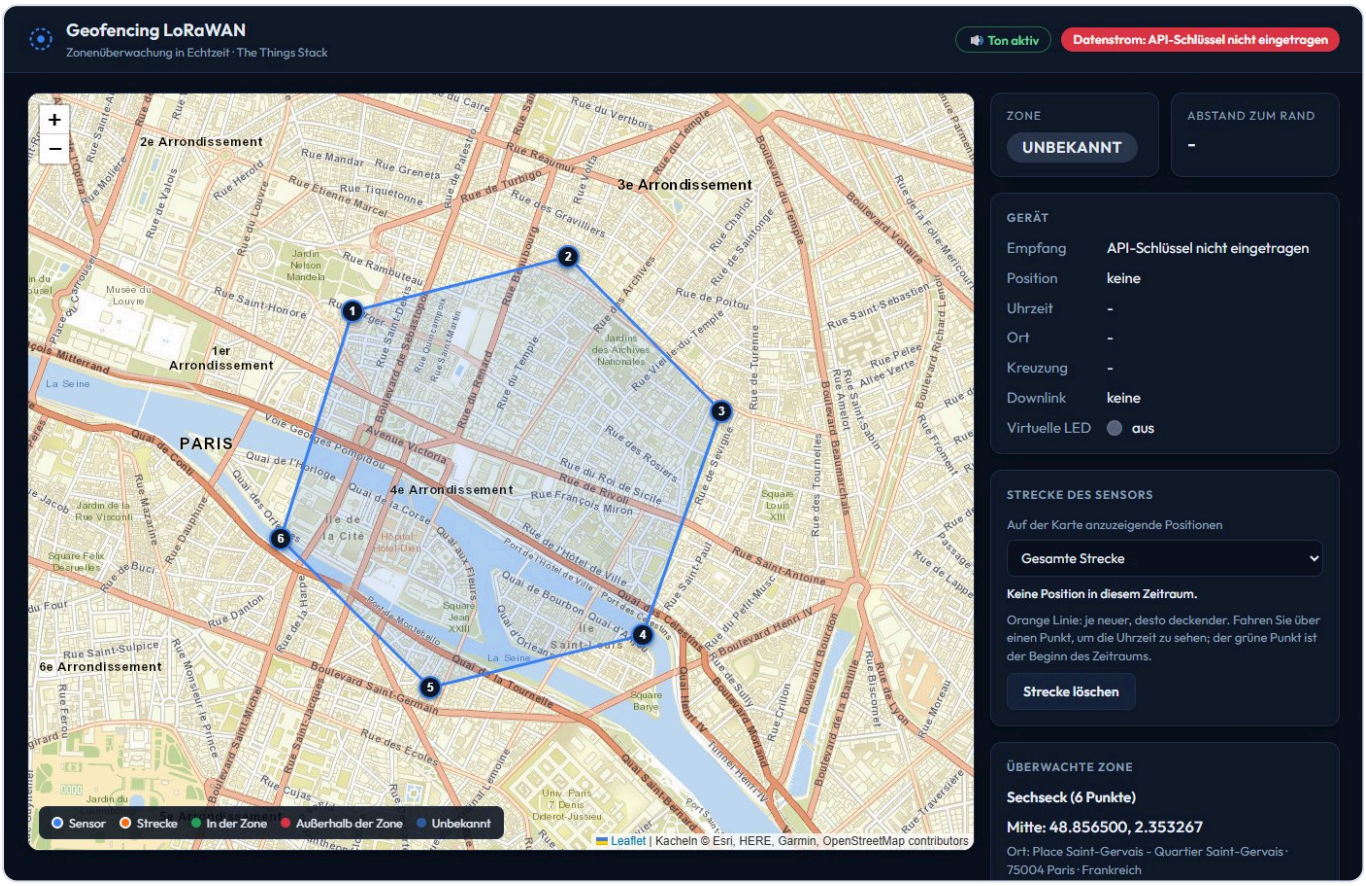
Ein vollständiger LoRaWAN-Rahmen hat 21 Byte: 8 Byte Position und 13 Byte Kopf und Signatur. Der Inhalt ist zwischen Gerät und Plattform verschlüsselt.

Klasse A: warum die LED einige Sekunden verzögert reagieren kann

Ein LoRaWAN-Gerät der Klasse A hört erst, nachdem es gesendet hat. Ein Downlink-Befehl wird daher erst nach dem **nächsten** Positionsversand übertragen. Bei einem Versand alle 30 s reagiert die LED höchstens 30 s nach dem Alarm. Ohne Gerät bleibt der Befehl einfach in der Warteschlange.

4. Die Weboberfläche

Eine einzige Seite im dunklen Design, lesbar am Computer und auf dem Handy. Die Karte bleibt sichtbar, während die rechte Seitenleiste scrollt.



Gesamtansicht: Polygonzone mit 6 Punkten (unregelmäßiges Sechseck), Zustand des Stroms, Anzeigen „Zone“ und „Abstand zum Rand“, Seitenleiste von Gerät und Strecke.

Was man sieht

- **Karte** mit der Zone (nummerierte Punkte), dem Sensor, seiner Strecke und einer Legende.
- **Anzeigen**: Zone (DRINNEN / DRAUSSEN), Abstand zum Rand, letzte Position, Uhrzeit, Adresse, nahe Kreuzung, Zustand des Downlinks, virtuelle LED.
- **Zustand des Stroms** und **Zustand des Tons** in der Kopfzeile.
- **Nachrichten** mit dem Patienten: ein Chat nach Art eines Messengers, beschrieben in Abschnitt 10.
- **Protokoll** mit Zeitstempel aller Ereignisse (Empfänge, Alarme, SMS, Fehler).

5. Die Kontrollzone als Polygon

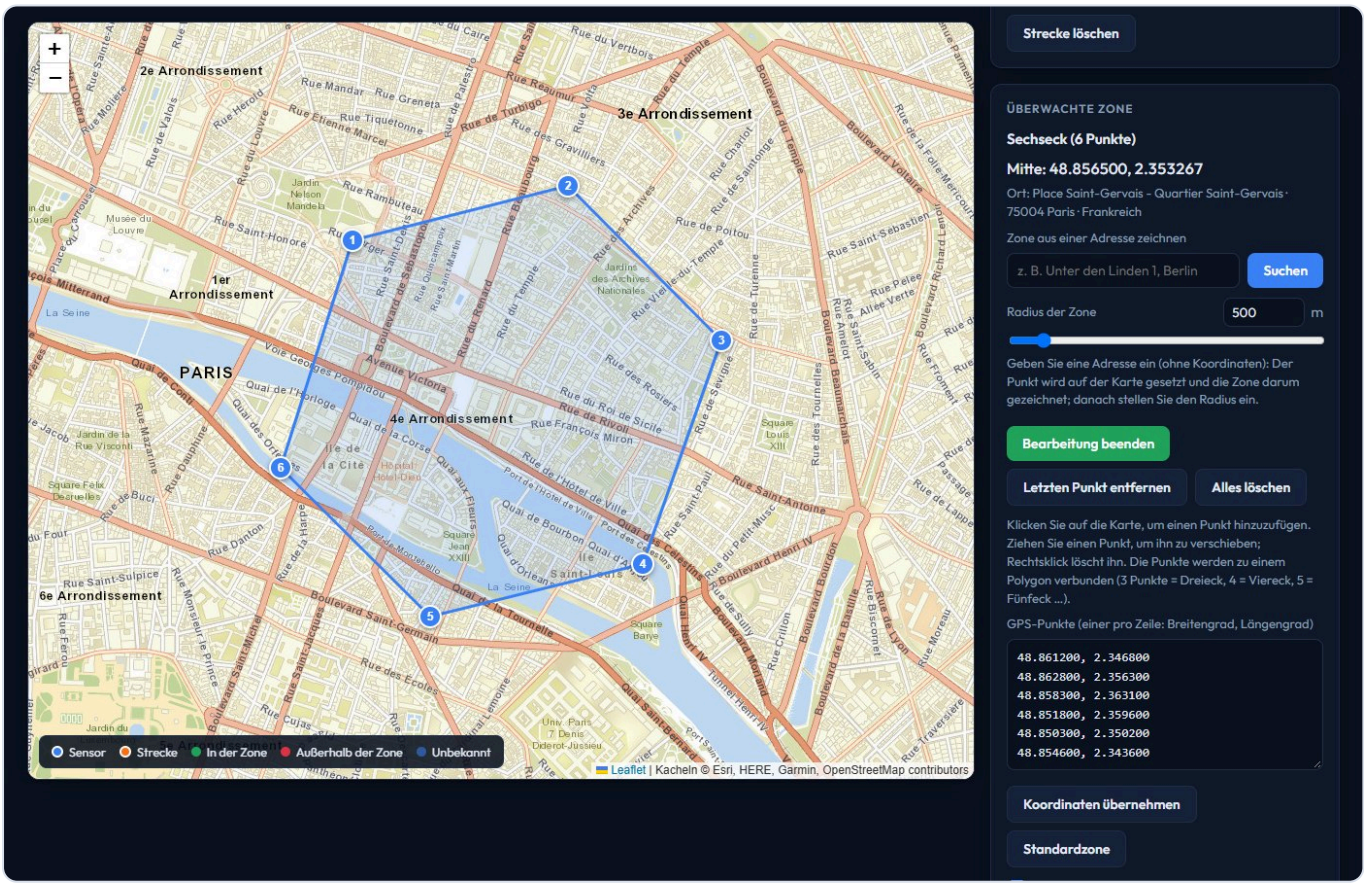
Die Zone wird durch eine Menge von GPS-Punkten definiert. Das System verbindet sie zu einer geschlossenen Form: 3 Punkte ergeben ein Dreieck, 4 ein Viereck, 5 ein Fünfeck und so weiter.

Die Zone definieren

- **Klick auf die Karte**, um einen Punkt hinzuzufügen; einen Punkt ziehen, um ihn zu verschieben; Rechtsklick löscht ihn.
- **Eingefügte Koordinaten**: eine Zeile „Breitengrad, Längengrad" pro Punkt.
- **Adresse + Radius**: Man tippt eine Adresse ohne Koordinaten; der Punkt wird gefunden (Dienst Nominatim von OpenStreetMap) und die Zone als Kreis aus 24 Punkten darum gezeichnet, mit einem Radius von 50 m bis 5 km (500 m standardmäßig). Weitere mögliche Treffer werden angeboten.
- **Standardzone** in der Konfigurationsdatei.
- Zone **gesperrt** außerhalb des Bearbeitungsmodus und im Browser gespeichert.
- Weniger als 3 Punkte: Zone „nicht definiert", kein Alarm.

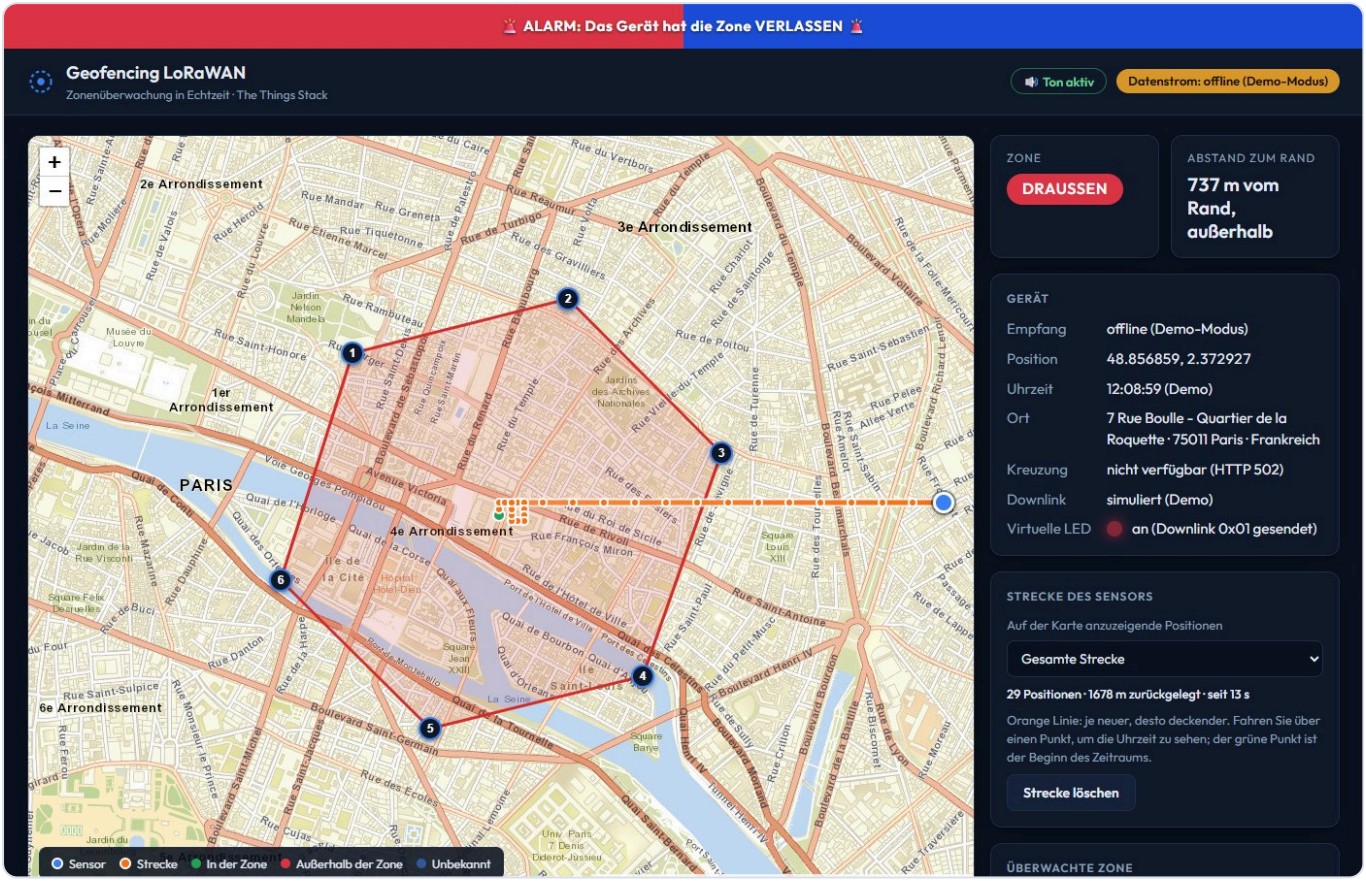
Den Austritt erkennen

- Die Punkte werden nach Winkel um die Mitte verbunden: keine sich kreuzenden Seiten, egal in welcher Reihenfolge eingegeben wurde.
- Zugehörigkeitstest per Strahlwurf; Abstand zum Rand in Metern berechnet (positiv innen, negativ außen).
- **Hysterese von 5 m**: Man muss den Rand um mehr als 5 m überschreiten, um den Zustand zu wechseln; das vermeidet Alarme in Serie, wenn das GPS schwankt.
- Ein Alarm nur bei einer **Zustandsänderung**.



Bearbeitungsmodus: nummerierte Punkte, Koordinatenfeld, Option für die automatische Reihenfolge der Punkte.

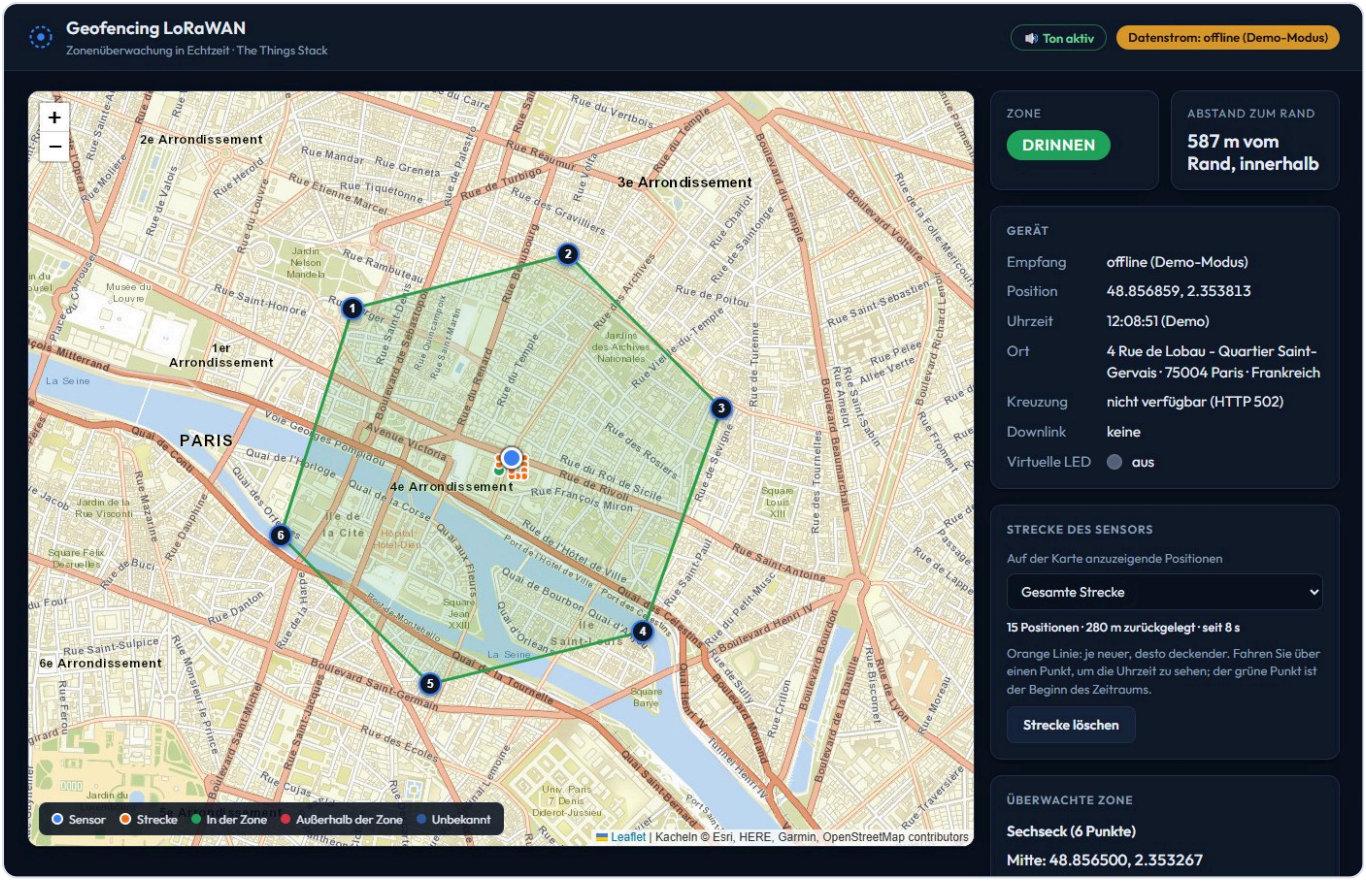
6. Die Alarme auf dem Bildschirm, per Ton und zum Gerät



Verlassen der Zone: blau-rotes Alarmbanner, Zone „DRAUSSEN", Abstand zum Rand, virtuelle LED an und Strecke bis zum Austrittspunkt.

Alarm	Verhalten
Banner	Rot und Blau im Wechsel beim Austritt; grünes Banner von 10 s bei der Rückkehr.
Ton	3 tiefe Pieptöne beim Austritt, 2 hohe bei der Rückkehr. Chrome blockiert den Ton, solange nicht auf die Seite geklickt wurde: Eine Anzeige „Ton aus / aktiv" zeigt das, und der wartende Alarm wird beim ersten Klick wiedergegeben.
Virtuelle LED	Grau im Ruhezustand, rot blinkend nach dem Befehl „einschalten".
Downlink	HTTPS-Aufruf an die Plattform (Byte 0x01 beim Austritt, 0x00 bei der Rückkehr), auf einen Versand pro Minute begrenzt; ein zu früher Befehl wird verzögert, nicht verworfen.
Demo-Modus	Der Sensor wird mit der Maus oder der Tastatur bewegt, ohne etwas Echtes zu senden: praktisch für eine Präsentation.

7. Strecke des Sensors und echte Orte



Vom Sensor zurückgelegte Strecke: orange Linie mit weißem Rand, Punkte mit Zeitstempel, Adresse des aktuellen Ortes.

Strecke, wie bei einer Taxi-App

- Dicke Linie hinter dem Sensor, blasser bei älteren Positionen.
- Einstellbare Dauer: 10 s, 30 s, 1 min, 5 min, 30 min, 1 h oder die gesamte Strecke.
- Grüner Punkt = Start; Uhrzeit beim Darüberfahren; zurückgelegte Strecke.
- Bis zu 5 000 Positionen werden behalten.

Die Linie verbindet die GPS-Punkte geradlinig; sie folgt nicht den Straßen.

Echte Orte

- **Adresse und Stadt** des Sensors und der Zonenmitte (Dienst Nominatim von OpenStreetMap).
- **Nächste Kreuzung** im Umkreis von 150 m (Dienst Overpass).
- Anfragen mit Abstand und zwischengespeichert, um die Grenzen der Dienste einzuhalten.

Die Kreuzung konnte auf dem echten Dienst nicht geprüft werden (Server bei den Versuchen überlastet).

8. Die SMS-Alarme

Bei jedem Austritt und danach bei jeder Rückkehr geht eine SMS an die konfigurierten Nummern. Beide Nachrichten sind standardmäßig aktiviert.

Austritt

```
ALARM: Ihr Patient hat gerade seine Kontrollzone verlassen (123 m vom Rand). 10:04 https://maps.google.com/?q=48.85660,2.35220
```

Rückkehr

```
INFO: Ihr Patient ist gerade in seine Kontrollzone zurueckgekehrt. 10:12 https://maps.google.com/?q=48.85660,2.35220
```

Sonderfall

```
ALARM: Ihr Patient befindet sich ausserhalb seiner Kontrollzone (11028 m vom Rand). 10:04 https://maps.google.com/?q=…
```

Wenn schon die allererste Messung außerhalb der Zone liegt.

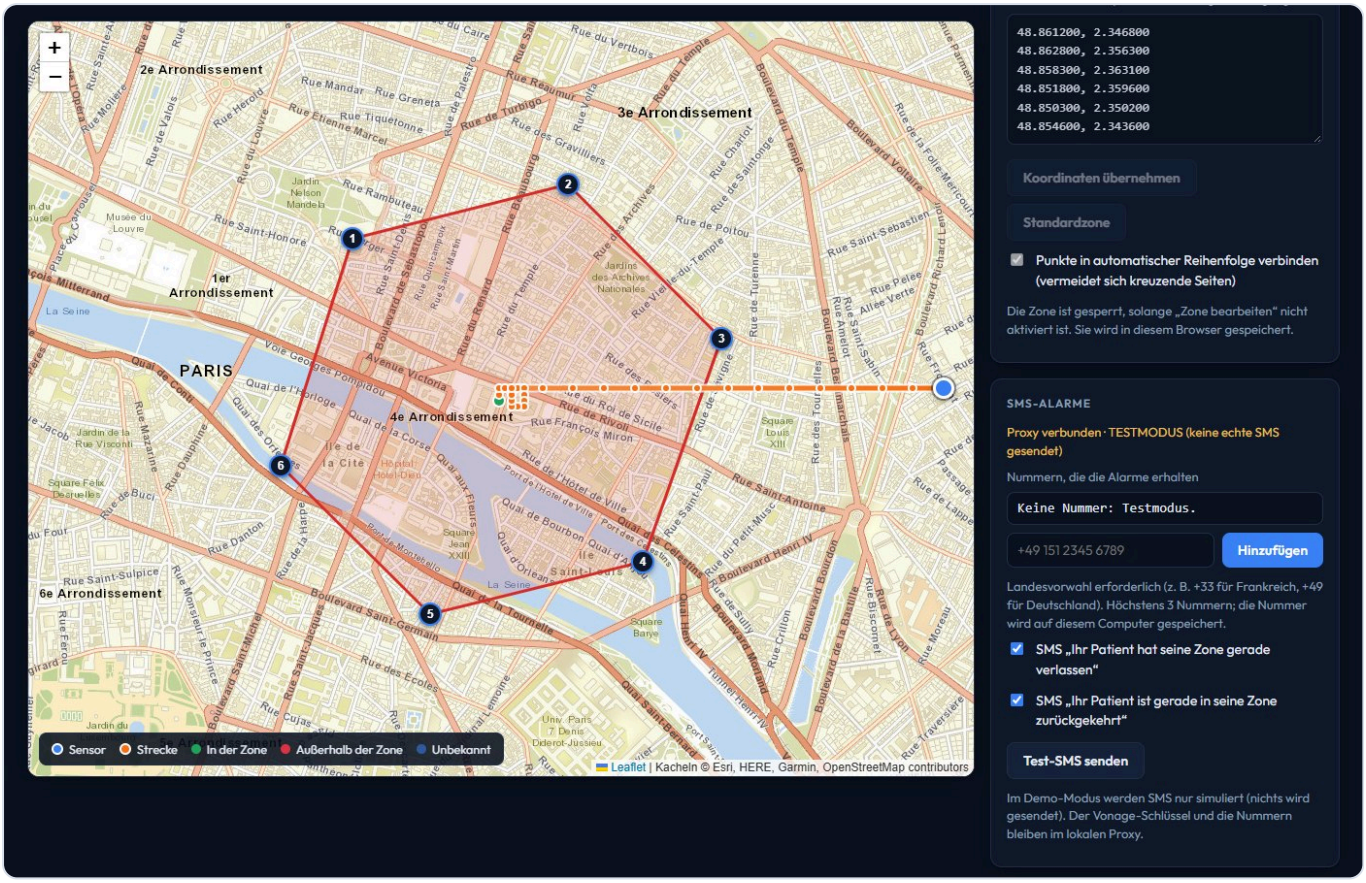
Die Nachrichten enthalten **keine Umlaute** (ä wird zu ae usw.), damit sie in eine einzige SMS von 160 Zeichen passen (ein einziges Zeichen außerhalb des GSM-Alphabets senkt die Grenze auf 70). Sie sind 113 bis 131 Zeichen lang. Der Link öffnet die Karte an der Position des Patienten. Die Formel „Ihr Patient“ lässt sich in der Konfiguration des Proxys durch einen Namen ersetzen.

Warum ein kleines lokales Programm

- Der Anbieter verlangt einen Schlüssel und ein Geheimnis: In die Seite gelegt, wären sie für alle lesbar, und jede SMS kostet Geld.
- Seine API-Adressen antworten nicht auf Anfragen eines Browsers.
- Der Proxy behält das Geheimnis und **setzt den Text selbst zusammen**: Die Seite kann die Nachricht nicht schreiben. Sie darf nur **eine Nummer hinzufügen oder entfernen** (höchstens 3) und sieht nie eine Nummer im Klartext, nur ihre letzten 3 Ziffern.

Schutzmaßnahme	Wert
Zugriff	Nur vom lokalen Rechner (127.0.0.1 und ::1); nur lokale Seiten
Abstand zwischen zwei SMS	Mindestens 30 s (einstellbar)
Limit	Auch nach einem Neustart gespeichert, einstellbar (`DAILY_MAX`): standardmäßig 10, **50 in der Testphase**; im Produktivbetrieb wieder zu senken (Schutz gegen eine Alarmschleife)
Nummern	In der Seite verwaltet (Abschnitt „SMS-Alarme“), in `tools/sms-recipients.json` gespeichert (von git und den Installern ausgeschlossen), haben Vorrang vor `SMS_TO` ; Zugriff nur für lokale Seiten
Wegen des Abstands abgelehnter Alarm	Eine wegen zu kurzen Abstands abgelehnte SMS geht nicht verloren: Die Seite sendet sie nach Ablauf der Wartezeit erneut, außer die Lage hat sich geändert; der Proxy protokolliert die Ablehnungen
Testmodus	Ohne Zugangsdaten wird der Text angezeigt, ohne etwas zu senden
Zustellverfolgung	15 s nach jedem Versand liest die Seite das Protokoll des Anbieters und zeigt das tatsächliche Ergebnis (zugestellt, abgelehnt mit Grund, ausstehend)

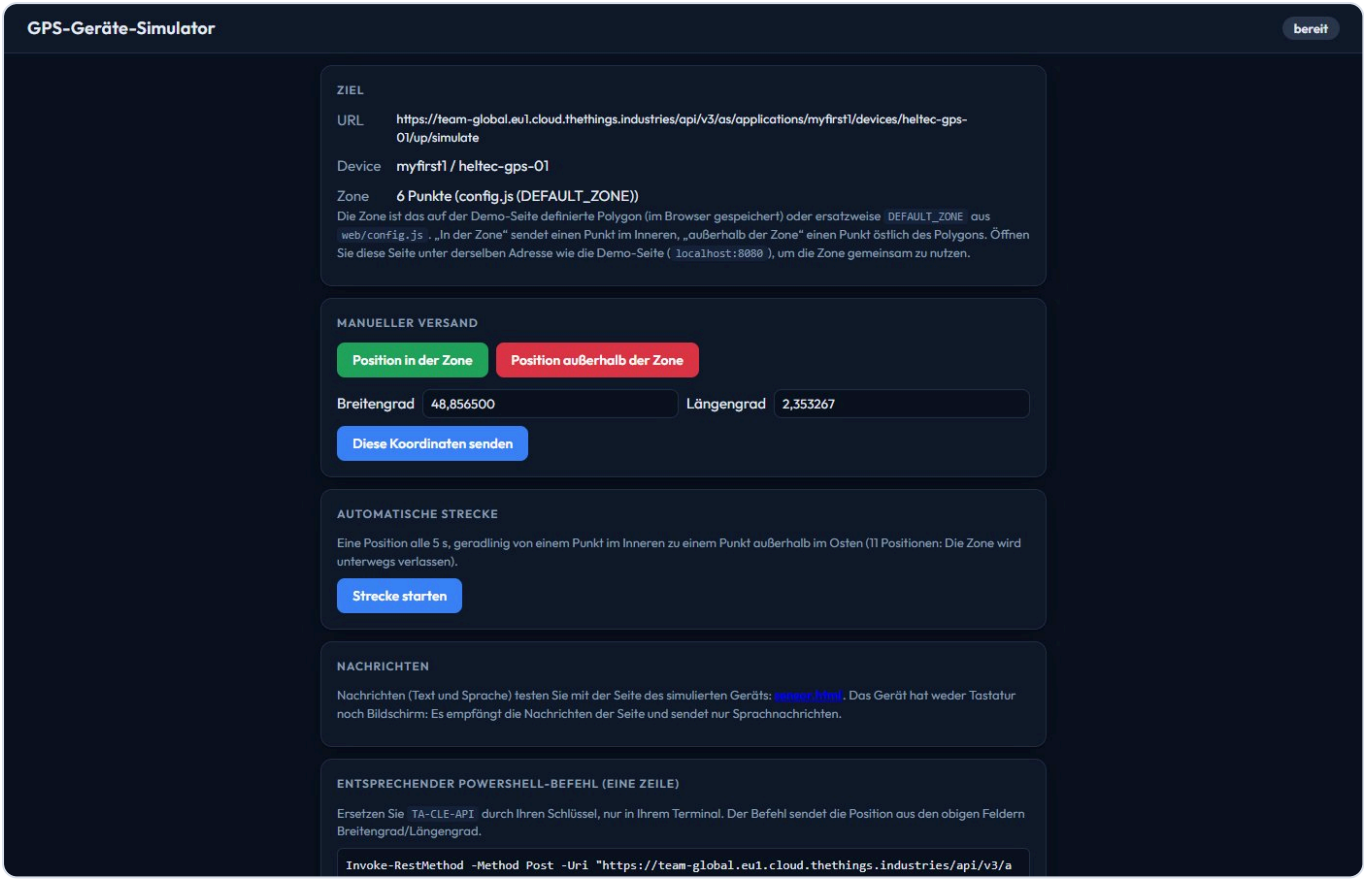
Ablauf eines SMS-Alarms



Abschnitt „SMS-Alarme": Zustand des Proxys, Nummern, Kästchen Austritt und Rückkehr, Test-Schaltfläche, Zustellergebnis und Protokoll. (Im Demo-Modus werden die SMS simuliert: Das Protokoll zeigt es an.)

#	Schritt
1	Die Seite erkennt die Zustandsänderung (Austritt oder Rückkehr) mit der Toleranz von 5 m.
2	Sie fordert beim lokalen Proxy den Versand an und gibt ihm nur das Ereignis, die Position und den Abstand zum Rand.
3	Der Proxy prüft den Mindestabstand und das Tageslimit, setzt den Text zusammen und ruft den Anbieter mit dem Geheimnis auf.
4	Der Anbieter antwortet „angenommen": Das Protokoll zeigt es an, mit der Zahl der heute verbleibenden SMS.
5	15 s später liest die Seite das tatsächliche Zustellergebnis und zeigt es grün (zugestellt), rot (abgelehnt, mit Grund) oder orange (ausstehend) an.

9. Der Simulator: ohne Hardware testen



tools/simulate.html: Versand von Positionen „in der Zone", „außerhalb der Zone", freie Koordinaten und automatische Strecke.

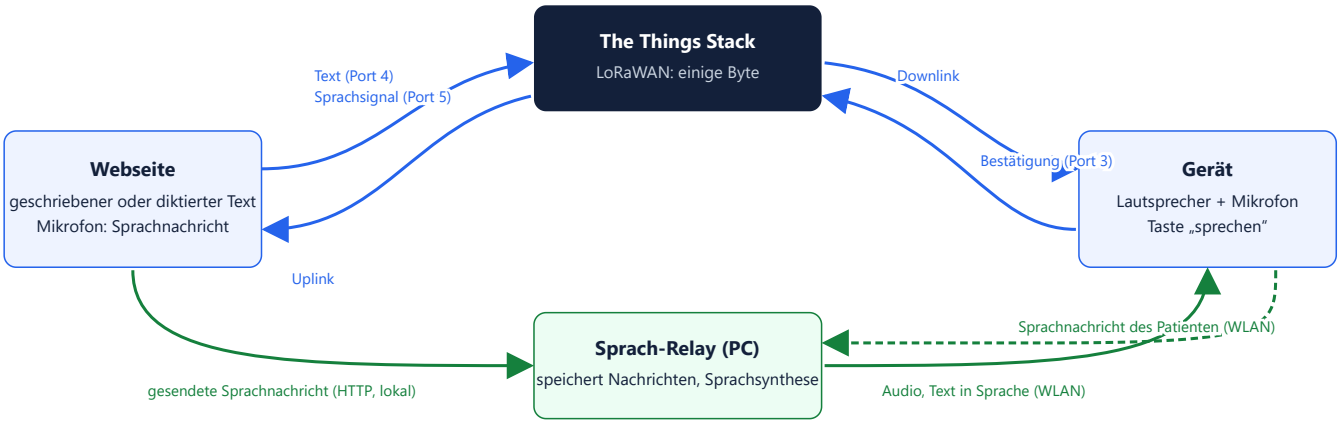
- Sendet echte simulierte Uplinks an die Plattform (gleiches 8-Byte-Format wie das künftige Gerät).
- Nutzt die auf der Demo-Seite definierte Zone: Der Punkt „in der Zone" liegt im Inneren des Polygons, der Punkt „außerhalb der Zone" östlich davon.
- **Automatische Strecke**: 11 Positionen, eine alle 5 s, die den Rand der Zone überqueren.
- Zeigt den entsprechenden PowerShell-Befehl und, bei einem Fehler 403, die dem Schlüssel hinzuzufügenden Rechte.
- Ein simulierter Uplink muss die Funkeinstellungen (`settings`) enthalten: Ohne sie antwortet die Plattform „400: settings erforderlich". Beide Simulationsseiten fügen sie hinzu.
- Für die **Nachrichten** und die **Karte** des Geräts siehe die Seite „simuliertes Gerät" (Abschnitt 11).

Vollständiges Testszenario, ohne Hardware

#	Schritt	Erwartetes Ergebnis
1	Gerät und Formatter auf der Plattform einrichten	Dekodierung in „Live data" sichtbar
2	Lokalen Server starten und die Seite öffnen	„Ereignisstrom aktiv"
3	Eine Position in der Zone senden	Sensor angezeigt, Zone „DRINNEN"
4	Eine Position außerhalb der Zone senden	Alarm, Ton, LED, SMS „Austritt"
5	In die Zone zurückkehren	Grünes Banner, SMS „Rückkehr"

10. Nachrichten zwischen Seite und Gerät

> **Das Gerät hat weder Tastatur noch Bildschirm.** Es kann daher weder Text schreiben noch anzeigen. Die Seite kann ihm einen **Text** senden (den das Gerät in Sprache umwandelt) oder eine **Sprachnachricht**. Das Gerät antwortet nur mit **Sprachnachrichten**.



Warum die Stimme nicht über LoRaWAN läuft

LoRaWAN transportiert 51 bis 222 Byte pro Nachricht, mit 1 % Sendezeit und sehr wenigen Nachrichten abwärts. Eine Sekunde Sprache wiegt hunderte Byte: unmöglich. Ein **kurzer Text** passt dagegen in eine Nachricht. Die Stimme läuft daher über das **WLAN** des Geräts und einen kleinen lokalen Dienst, das **Sprach-Relay**.

Richtung	Was	Worüber
Seite → Gerät	**Text** (höchstens 49 Byte)	LoRaWAN, Port 4; das Gerät **wandelt ihn in Sprache um** (lokale Sprachsynthese des Relays) und spricht ihn
Seite → Gerät	**Sprachnachricht** (höchstens 10 s)	Audio: WLAN, über das Relay. Das Gerät wird durch einen Downlink von 1 Byte (Port 5) benachrichtigt
Gerät → Seite	**Sprachnachricht** des Patienten	WLAN, über das Relay (nie über LoRaWAN); wird auf der Seite **automatisch abgespielt**
Gerät → Seite	Bestätigung „laut gesprochen"	LoRaWAN, Port 3 (1 Byte)

10 (Fortsetzung). Die Nachrichten der Seite



Abschnitt „Sprachnachrichten" der Seite: grüne Blasen gesendet, graue empfangen, Haken, grünes Mikrofon.

Schreiben, diktieren oder sprechen

- **Schreiben** eines Textes von höchstens 49 Byte (ein Umlaut zählt 2). Der Zähler zeigt es an.
- **Diktieren** mit dem kleinen Symbol im Textfeld: Die Spracherkennung von Chrome schreibt den Text, den man vor dem Senden noch einmal liest. Sie bleibt bis zu einem Klick aktiv.
- **Sprechen** mit dem grünen Mikrofon: ein Klick zum Beginnen, ein Klick auf „Senden" zum Beenden; höchstens 10 s in der Testphase.

Die Haken

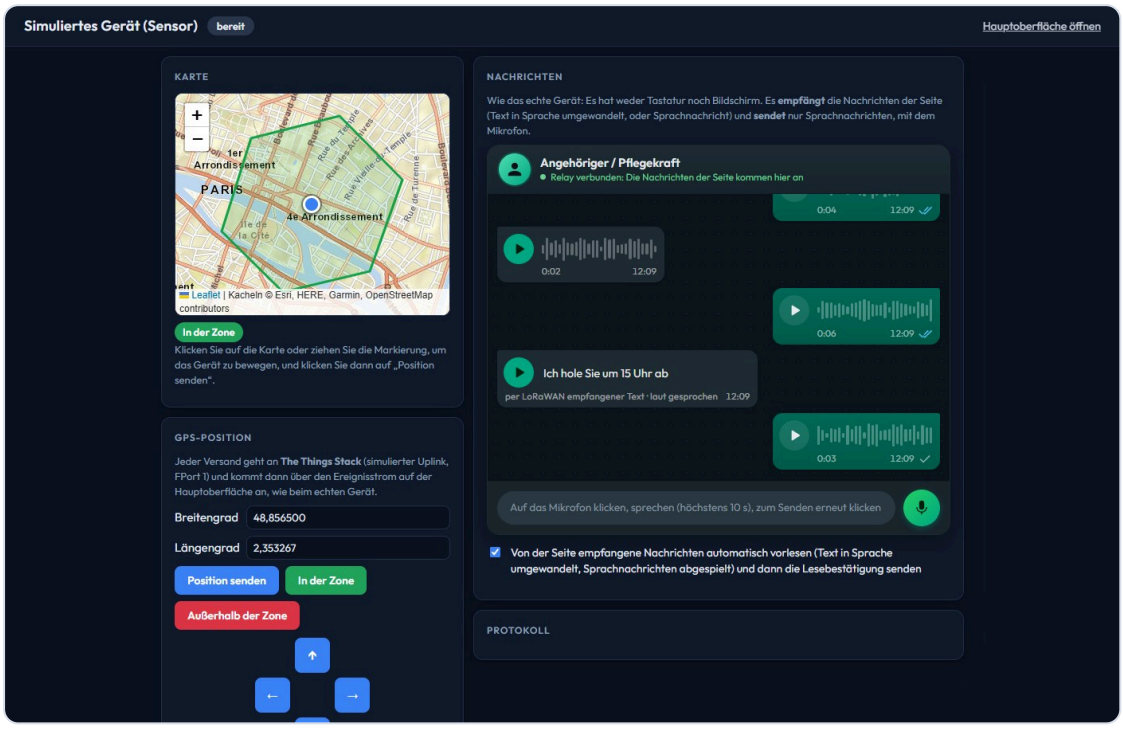
Haken	Bedeutung
Uhr	Wartet auf Versand (höchstens ein Downlink pro Minute)
✓	Gesendet, oder an das simulierte Gerät zugestellt
✓✓	Vom Gerät empfangen
Blaue ✓✓	Laut gesprochen (Text) oder angehört (Sprachnachricht)
Rotes Dreieck	Nicht gesendet

Grenzen und Regeln

- Text: 49 Byte. Sprachnachricht: 10 s (WAV 8 kHz, mono); das Relay akzeptiert bis zu 30 s und 1 MB.
- 20 Sprachnachrichten und 200 Texte pro Stunde; Nachrichten nach 7 Tagen gelöscht.
- Mehrere Nachrichten hintereinander: Sie warten, bis sie an der Reihe sind (die LED behält den Vorrang).
- Ohne echtes Gerät lehnt die Plattform den Downlink ab („no_device_session"): Ist die Seite des simulierten Geräts geöffnet, wird ihr die Nachricht über das Relay zugestellt.

11. Die Seite des simulierten Geräts (Sensor)

Eine eigene Seite, `tools/sensor.html`, übernimmt die Rolle des Geräts. Sie sendet **echte simulierte Uplinks** an The Things Stack, die die Hauptoberfläche danach über den Ereignisstrom empfängt, genau wie mit dem künftigen Gerät.



Seite des simulierten Geräts: Karte mit der Zone und der Markierung des Geräts (zum Ziehen oder Anklicken), GPS-Position und Nachrichten. Hier ein Beispielaustausch.

Was sie tut

- **Karte**: die Zone und die Markierung; „In der Zone" oder „Außerhalb der Zone" erscheint unter der Karte.
- **Position**: Koordinaten, Pfeile (20, 100 oder 500 m), Versand bei jeder Bewegung (Kästchen standardmäßig angehakt) und automatischer Versand alle 30 s wie das echte Gerät (optional).
- **Mikrofon**: Sprachnachricht von höchstens 10 s an die Seite, über das WLAN-Relay.
- **Empfangene Nachrichten**: Text in Sprache umgewandelt und vorgelesen; Sprachnachricht abgespielt; dann Lesebestätigung gesendet.
- Der Verlauf bleibt erhalten und stammt vom Relay.

Zu wissen

- Der Decoder muss in der Konsole aktuell sein, und der Schlüssel braucht das Recht „Write uplink application traffic".
- Der Versand nach jeder Bewegung ist **standardmäßig angehakt**: Das Verschieben der Markierung aus der Zone löst einen echten SMS-Alarm aus, wenn die Hauptoberfläche nicht im Demo-Modus ist.
- Die Hauptoberfläche muss mit abgewähltem Demo-Modus geöffnet sein, um die Uplinks zu empfangen.
- Jede geöffnete Seite, die die Plattform abhört, reagiert auf die Alarme: Öffnen Sie nur eine.

12. Das Projekt auf einem anderen Rechner installieren

Ziel: Eine Person, mit oder ohne Entwicklerkenntnisse, holt den Code von GitHub und lässt ihn in wenigen Minuten bei sich laufen, mit einem einzigen Befehl zum Kopieren.

Der Befehl

Windows (PowerShell)

```
irm https://raw.githubusercontent.com/Arseine1/geolocalise-deutsch/main/install.ps1 | iex
```

macOS oder Linux (Terminal)

```
curl -fsSL https://raw.githubusercontent.com/Arseine1/geolocalise-deutsch/main/install.sh | bash
```

Was der Installer tut

1
Prüft Node.js
Einziges nötiges Programm. Unter Windows bietet er die Installation an.

2
Lädt herunter
Das Projekt von GitHub, ohne dass git nötig ist.

3
Konfiguriert
Einige Fragen; alles lässt sich mit Enter überspringen.

4
Startet
Server, SMS-Alarme und Öffnen der Seite.

Bedarf	Lösung
Keine Kommandozeile	Schaltfläche „Download ZIP“ auf GitHub, entpacken, Doppelklick auf `Starten.bat` (Windows) oder `./starten.sh`
Mit git	`git clone https://github.com/Arseine1/geolocalise-deutsch.git`, dann `node start.js`
Später neu starten	Doppelklick auf `Starten.bat` oder `node start.js`
Aktualisieren	Denselben Befehl erneut ausführen: Der Code wird ersetzt, **Schlüssel, Nummern und Einstellungen bleiben erhalten**
Deinstallieren	Den Ordner `geolocalise-deutsch` löschen

Was der Konfigurationsassistent fragt

Adresse, Bereich, Anwendung, Gerät und Webhook von The Things Stack; API-Schlüssel (Eingabe verdeckt); Vonage-Zugangsdaten und Telefonnummer (optional). Ein optionaler Schritt richtet die Sprachnachrichten ein (nächste Seite). Eine als „+49 151 2345 6789“ eingegebene Nummer wird automatisch umgewandelt. Ganz ohne Schlüssel funktioniert der **Demo-Modus** trotzdem, und SMS werden nur angezeigt. Die Geheimnisse bleiben auf dem Computer, in drei von git ausgeschlossenen Dateien.

> **Wichtig: Ist das GitHub-Repository privat,** antwortet es nicht angemeldeten Personen mit „404“: Der obige Befehl funktioniert dann erst, wenn das Repository **öffentlich** ist. Es enthält keine Geheimnisse. Soll es privat bleiben, akzeptiert der Installer ein **schreibgeschütztes GitHub-Token** (Variable `GEO_TOKEN`), oder die Person wird als Mitarbeiter hinzugefügt.

12 (Fortsetzung 1). Tests der Installation

Die Installation wurde auf einem leeren Ordner ausprobiert, mit Archiven, die die von GitHub nachbilden, und mit den echten Installern. Die Tests wurden für die deutsche Fassung erneut ausgeführt.

#	Test	Ergebnis
1	Windows-Installer (PowerShell) auf einem leeren Ordner: Dateien, Konfiguration angelegt, Exit-Code 0	**Bestanden**
2	Aktualisierung einer bestehenden Installation: `voice.env` und `config.js` byte-gleich erhalten, Code ersetzt	**Bestanden**
3	macOS/Linux-Installer (unter Git Bash ausprobiert) auf einem leeren Ordner: Dateien, Konfiguration, Token erzeugt und nicht angezeigt	**Bestanden**
4	Automatische Tests des Relays (`node tools/test-voice.js`) mit der deutschen Fassung, inklusive echter Sprachsynthese	**74 bestanden**
5	Deutsche SMS-Texte im Testmodus des Proxys: Austritt, Rückkehr, erste Position außerhalb der Zone (jeweils unter 160 Zeichen, ohne Umlaute)	**Bestanden**
6	Deutsche Oberfläche im Browser (Demo-Modus): Adresse „Brandenburger Tor, Berlin" gesucht, Zone gezeichnet, Text gesendet, Alarm ausgelöst	**Bestanden**
7	Echter Download vom öffentlichen Repository mit beiden Befehlen (`curl … | bash` und `irm … | iex`): Dateien, Konfiguration, Exit-Code 0	**Bestanden**
8	Relay-Tests unter Linux (Ubuntu in WSL) mit der heruntergeladenen Kopie, Sprachsynthese mit `espeak-ng` (deutsche Stimme)	**74 bestanden**
9	Ein echter deutscher Alarm-SMS (Austritt) über Vonage an die konfigurierte Nummer: angenommen und vom Netzbetreiber als „Zugestellt" gemeldet	**Bestanden**
10	Sketch des Geräts mit g++ (`-Wall -fsyntax-only`) gegen Attrappen der Bibliotheken übersetzt, in allen Varianten; Aufrufe von RadioLib mit den echten Headern 7.8.1 verglichen	**Bestanden**

Was noch nicht geprüft ist

- **macOS**: nicht getestet (Befehl `say` , Stimme Anna). Linux wurde nur in WSL (Ubuntu) und nur für das Relay geprüft, nicht der Installer auf einer echten Linux-Maschine.
- **Die automatische Installation von Node.js** auf einem Rechner ohne Node.js: nicht ausgeführt, um diesen Rechner nicht zu verändern.
- **Der französische Text im Original** wurde nicht geändert; die deutsche Fassung ist eine eigene Kopie, die nicht automatisch mit dem Original synchron bleibt.

12 (Fortsetzung 2). Was die Nachrichten für die Installation ändern

Die Installation mit einem Befehl bleibt dieselbe. Die Nachrichten bringen einige Einstellungen und Voraussetzungen mit, hier zusammengefasst.

Thema	Was sich ändert
Konfigurationsassistent	Neuer Schritt „Sprachnachrichten“: aktivieren oder nicht, das Relay für das lokale Netz öffnen (standardmäßig nein), Aufbewahrungsdauer (7 Tage). Er legt `tools/voice.env` mit einem erzeugten, nie angezeigten Token an; Datei von git ausgeschlossen.
Aktualisierung	Beide Installer behalten `web/config.js`, `tools/sms-proxy.env`, `tools/voice.env`, `tools/sms-recipients.json` (in der Seite eingegebene Nummern) und `secrets.h` des Geräts. Die bereits gespeicherten Nachrichten (`data/voice/`) bleiben unberührt.
Konsole The Things Stack	Nach einer Aktualisierung, die den Ordner `ttn/` ändert, **Decoder (Tab Uplink) und Encoder (Tab Downlink)** erneut in „Payload formatters“ des Geräts kopieren. Der API-Schlüssel braucht für die Seite des simulierten Geräts das Recht „Write uplink application traffic“.
Browser	Chrome, unter der Adresse `http://localhost:8080` (weder IP-Adresse noch geöffnete Datei): Mikrofon und Diktat verlangen das. Das Diktat braucht Internet.
Sprachsynthese	Text wird von der Engine des Systems in Sprache umgewandelt: unter Windows und macOS (Befehl `say`) enthalten; unter Linux `espeak-ng` installieren. Ohne Engine nutzt die Seite des simulierten Geräts die Stimme des Browsers.
Echtes Gerät (später)	`VOICE_LAN=1` (Port 8081), den Port in der Firewall erlauben, das Token in `secrets.h` der Firmware übernehmen. Unverschlüsseltes HTTP: nur für ein vertrauenswürdiges Netz.

13. Der Code des Geräts (bereit für die Hardware)

Arduino-Programm für eine Platine **Heltec WiFi LoRa 32 V3** mit einem GPS-Modul **NEO-6M**. Es ist geschrieben und durchgesehen, wurde aber **nie kompiliert oder auf einer Platine ausgeführt**.

Funktion	Einzelheiten
Netz	LoRaWAN, OTAA-Aktivierung, Band EU868 (Bibliothek RadioLib 7.8.1)
Senden	Eine Position von 8 Byte alle 30 s auf Port 1, **nur wenn der GPS-Fix gültig ist**; Lesebestätigung eines Textes auf Port 3
Empfang	Port 2: 0x01 = LED blinkt, 0x00 = LED aus; Port 4: Text, der laut zu sprechen ist; Port 5: eine Sprachnachricht wartet auf dem Relay
Stimme	I2S-Mikrofon INMP441 und Verstärker MAX98357A mit Lautsprecher; Taste „sprechen“ (höchstens 10 s, fortlaufend per WLAN gesendet); Wiedergabe der Sprachsynthese und der Sprachnachrichten der Seite, vom Relay heruntergeladen
Testmodus	`TEST_OHNE_GPS` : simulierte Positionen ohne GPS-Modul
Bibliotheken	RadioLib 7.8.1, TinyGPSPlus 1.0.3, Board-Unterstützung ESP32 3.3.12

> **Am Tag des Aufbaus zu prüfen:** die beiden Pins des GPS und die Pins von Mikrofon, Verstärker und Taste (gewählt, nicht vorgegeben), die TCXO-Spannung des Funkmoduls (1,8 V angenommen), das Vorhandensein eines in der Plattform registrierten Gateways und das Verhalten der Netzbetreiber je nach Land. **Der Sprachteil ist am wenigsten geprüft**: Die Namen der Audio-Schnittstelle (`ESP_I2S`) und die Form des Aufrufs zum Senden der Bestätigung sind aus dem Gedächtnis geschrieben und können beim Kompilieren Korrekturen erfordern. In der deutschen Fassung wurden außerdem die französischen Bezeichner des Sketches ins Deutsche umbenannt: Der Sketch übersetzt weiterhin ohne Fehler gegen Attrappen der Bibliotheken (belegt nur die Stimmigkeit der Namen), und die Aufrufe von RadioLib stimmen mit den echten Headern von Version 7.8.1 überein. Auf einer Platine wurde nichts kompiliert.

14. Stand der Arbeiten

Jedes Element ist danach eingestuft, was tatsächlich geprüft wurde.

Element	Status	Beleg oder Grenze
Gerät, Formatter, API-Schlüssel und Webhook auf der Plattform	**Validiert**	Von Ihnen angelegt; Dekodierung in „Live data" sichtbar
Empfang der simulierten Positionen in der Seite	**Validiert**	Nach Korrektur des Stroms bestätigt (Wiederverbindung und Nachholen)
Test-SMS (Vonage → Telefon)	**Validiert**	SMS auf einer deutschen Nummer empfangen
Polygonzone, Erkennung, Hysterese	**In Simulation getestet**	Dreieck, Viereck, Fünfeck; Abstände und Schwellen geprüft
Optische Alarme, Ton, virtuelle LED	**In Simulation getestet**	Der Ton hängt von einem Klick auf die Seite ab; nicht durchgängig mit dem Ohr bestätigt
Strecke, Adresse, Oberfläche	**In Simulation getestet**	Durch Aufnahmen und Messungen geprüft
SMS-Nachrichten, Schutzmaßnahmen und Zustellverfolgung des Proxys	**In Simulation getestet**	Text, Länge, Umlaute im Testmodus geprüft; falscher Anbieter; echtes Lesen des Protokolls bei einer abgelehnten SMS
Text- und Sprachnachrichten zwischen Seite und simuliertem Gerät, Bestätigung per Uplink	**In Simulation getestet**	Bestätigung als echter simulierter Uplink gesendet, dekodiert und von der Seite empfangen; 74 Tests des Relays; Mikrofon durch einen erzeugten Ton simuliert, kein echtes Mikrofon
Seite des simulierten Geräts (Karte, Position) und Diktat	**In Simulation getestet**	Uplinks von der Plattform akzeptiert (Code 200); Diktat mit einer falschen Erkennung geprüft
Aus einer Adresse gezeichnete Zone mit einstellbarem Radius	**In Simulation getestet**	Echte Suche (Nominatim) im Browser geprüft, auch mit der deutschen Oberfläche: Punkt gesetzt, 24 Punkte, Radius per Regler geändert; Demo-Modus
In der Seite geänderte SMS-Nummern	**In Simulation getestet**	Route des Proxys getestet (Hinzufügen, Entfernen, Duplikat, ungültige Nummer, Grenze von 3, abgelehnter Ursprung) auf einer isolierten Kopie; keine echte SMS
Nach einer Ablehnung wegen des Abstands erneut gesendete SMS; automatisches Abspielen der Sprachnachrichten	**Nicht getestet**	Code geschrieben, nur Syntax geprüft; die Wiedergabe hängt vom Browser ab (ein Klick kann nötig sein)
Installation mit einem Befehl (Windows und Git Bash)	**In Simulation getestet**	Beide Installer mit der deutschen Fassung auf einem leeren Ordner und als Aktualisierung bestanden, mit lokalem Archiv
Echter Download von GitHub (beide Befehle)	**Validiert**	Vom öffentlichen Repository ausgeführt, Exit-Code 0; macOS nicht ausprobiert
Echte SMS in deutscher Sprache	**Validiert**	Alarm-SMS „Austritt" angenommen und als zugestellt gemeldet (Lebara Germany); kein Test des Ablaufs Uplink → Alarm → SMS
Relay und Sprachsynthese unter Linux	**In Simulation getestet**	Ubuntu in WSL, `espeak-ng` mit deutscher Stimme; 74 Tests bestanden; keine echte Linux-Maschine, kein macOS
Echter Alarm, der eine SMS auslöst, durchgängig	**Nicht getestet**	Nächster Test mit dem Simulator
Von einem echten Gerät empfangener Downlink	**Nicht getestet**	Einreihung in „Live data" zu prüfen
Code des Geräts auf der Platine (Position und Stimme)	**Nicht getestet**	Nie auf einer Platine kompiliert; mit Attrappen übersetzbar, RadioLib-Aufrufe geprüft; der Audioteil ist am wenigsten geprüft
Nachrichten an ein echtes Gerät (Ports 4 und 5)	**Nicht getestet**	Ein dem Netz beigetretenes Gerät (OTAA) ist nötig
Nahe Kreuzung (echter Dienst)	**Nicht getestet**	Server bei den Versuchen überlastet
SMS nach Kamerun	**Blockiert**	Von einer Länderregel des Anbieters abgelehnt (Code 24)

15. Technische Entscheidungen und aufgetretene Hindernisse

Hindernis	Feststellung	Entscheidung
MQTT aus dem Browser	Kein MQTT-WebSocket auf den getesteten Ports	HTTP-Ereignisstrom der Plattform
Strom alle 15 s getrennt	Positionen gingen beim Wiederöffnen verloren	Sofortiges Wiederöffnen und Nachholen der verpassten Ereignisse
Kartenhintergrund blockiert	Der offizielle Server liefert „403 Access blocked“; ein anderer Anbieter verlangt einen Schlüssel	Esri-Hintergrund ohne Schlüssel, Anbieter konfigurierbar
SMS-Schlüssel in der Seite	Für alle lesbar, und die API ist für Browser geschlossen	Kleiner lokaler Proxy, der das Geheimnis behält
SMS „angenommen“, aber nicht empfangen	„Angenommen“ heißt nicht „zugestellt“	Automatische Zustellprüfung, mit dem Grund der Ablehnung
Kamerun abgelehnt	Länderregel des Betrugsschutzes des Anbieters (Code 24)	Versuch mit einer deutschen Nummer
Anderer erwogener Anbieter	Verlangt bei der Anmeldung eine Aufladung von 20 $	Rückkehr zum ersten Anbieter, der schon Guthaben hat
Komplizierte Installation	Python für die Seite, Node für die SMS, Konfiguration von Hand	Nur Node.js, integrierter Webserver, Konfigurationsassistent, Installer mit einem Befehl
Privates GitHub-Repository	Ein privates Repository antwortet nicht angemeldeten Personen mit „404“	Öffentliches Repository (am einfachsten) oder Lese-Token; beides wird unterstützt
Allgemeine Port-Einstellung	Die Variable „PORT“ wird von anderen Programmen genutzt und störte den Proxy	Projekteigene Einstellung: SMS_PROXY_PORT
Stummer Ton	Richtlinie des Browsers: Ein Klick ist nötig	Statusanzeige, Freischaltung beim ersten Klick, Alarm wird wiederholt
Stimme über LoRaWAN	51 Byte pro Nachricht und 1 % Sendezeit: Eine Sekunde Sprache ist unmöglich	Text per LoRaWAN, vom Gerät in Sprache umgewandelt; Audio per WLAN über ein lokales Relay
Gerät ohne Tastatur und Bildschirm	Es kann weder schreiben noch anzeigen	Es empfängt Text (in Sprache umgewandelt) und Sprachnachrichten und sendet nur Sprachnachrichten
Simulierter Uplink abgelehnt (400)	Die Plattform verlangt die Funkeinstellungen (`settings`)	Eine simulierte EU868-Datenrate in beiden Simulationsseiten ergänzt
Nachrichten „nicht gesendet“	Limit von 20 Nachrichten pro Stunde für Texte und Sprachnachrichten gemeinsam; Downlink ohne Gerät abgelehnt	Getrennte Zähler (20 Sprachnachrichten, 200 Texte); Zustellung über das Relay an die Seite des simulierten Geräts
Diktat bricht ab	Chrome beendet die Erkennung nach einigen Sekunden Stille	Automatischer Neustart, Text bleibt erhalten, Ende per Klick (in der Testphase höchstens 10 s)
Protokoll unter der Karte	Die oben angeheftete Karte lief über	Karte und Protokoll im Layout getrennt

16. Sicherheit und Datenschutz

Technische Sicherheit

- Der Schlüssel der Plattform ist in der Seite lesbar: **nur lokale Nutzung**, nach der Vorführung widerrufen, mit minimalen Rechten.
- Das Geheimnis des SMS-Anbieters bleibt in einer lokalen, vom Repository ausgeschlossenen Datei, die nur der Proxy liest.
- Der Proxy lauscht nur lokal und lehnt andere Ursprünge ab. Er lässt nun jede lokale Seite eine Nummer hinzufügen oder entfernen (höchstens 3): Keine externe Seite kann das.
- Limit und Mindestabstand gegen Sendungen in Serie.

Datenschutz

- Die Position einer Person ist ein sensibles Datum. Sie geht per SMS hinaus, also an den Anbieter und die Netzbetreiber.
- Die angezeigte Adresse entsteht, indem die Position an einen öffentlichen Geokodierungsdienst gesendet wird; auch die zum Zeichnen der Zone eingegebene Adresse wird ihm gesendet.
- Für einen echten Einsatz bei Patienten: Einwilligung, Zweck, Aufbewahrungsdauer und der für Gesundheitsdaten geltende Rahmen sind vor dem Einsatz festzulegen.

Nachrichten und Stimme. Eine Stimme und an eine überwachte Person gerichtete Nachrichten sind sensible personenbezogene Daten. Die Nachrichten bleiben auf dem Computer (Ordner `data/voice/`, von git ausgeschlossen und nie ausgeliefert) und werden nach 7 Tagen gelöscht. Der per LoRaWAN gesendete Text läuft wie die Position über die Plattform: Er darf keine sensiblen Angaben enthalten. Die Sprachsynthese ist lokal; nur das **Diktat** von Chrome sendet die Stimme an Google. Das Relay akzeptiert die Seite nur von `localhost`; das Gerät legt ein geheimes Token vor. Der Modus „lokales Netz“ (für ein echtes Gerät) nutzt unverschlüsseltes HTTP: nur für ein vertrauenswürdiges Netz. Die überwachte Person muss informiert werden, dass der Austausch aufbewahrt wird.

Erinnerung. Die bei den Versuchen verwendeten Schlüssel sind durch Arbeitsgespräche gelaufen: Es ist ratsam, sie vor jeder dauerhaften Nutzung neu zu erzeugen.

17. Nächste Schritte

#	Schritt	Nutzen
0	Den Code auf GitHub veröffentlichen (öffentliches Repository oder Lese-Token)	Die Installation mit einem Befehl ermöglichen und in echt testen
1	Einen echten Alarm mit dem Simulator testen	„Uplink → Alarm → SMS“ durchgängig validieren
2	Ein Gateway und die Platine beschaffen	Die Simulation verlassen; den Code des Geräts kompilieren und testen
2 bis	Die Stimme auf der echten Platine ausprobieren	Mikrofon, Verstärker und Taste anschließen; den Audioteil des Codes beim Kompilieren korrigieren
2 ter	Die Sprachnachrichten verlängern (heute 10 s)	Eine Lösung für Dauer und Speicher des Geräts finden, das keinen zusätzlichen Speicher hat
3	SMS auch bei geschlossener Seite senden	Die Positionen von einem kleinen Server empfangen lassen, der die Zonenberechnung wiederholt und die SMS sendet
4	Linie, die den Straßen folgt	Die Strecke mit einem Straßen-Matching-Dienst lesbarer machen
5	Dauerhafter Verlauf	Die Strecken über das Neuladen der Seite hinaus behalten
6	Rechtlicher Rahmen	Einwilligung und Datenschutz vor einem Einsatz bei Patienten
7	Diktat zu 100 % lokal und verschlüsselte Verbindung	Die Stimme nicht mehr an Google senden; das Relay für den Modus „lokales Netz“ auf HTTPS umstellen

18. Vorgeschlagener Präsentationsplan

Jeder Abschnitt dieses Dokuments kann zu einer Folie werden:

Folie	Titel	Zu übernehmender Inhalt
1	Geofencing LoRaWAN	Titel, Untertitel, Ziel (Abschnitt 1)
2	Der Bedarf	Einen Angehörigen benachrichtigen, wenn ein Patient seine Zone verlässt
3	Architektur	Schema aus Abschnitt 2
4	Der Rahmen von 8 Byte	Schema aus Abschnitt 3
5	Die Oberfläche	Aufnahme aus Abschnitt 4
6	Zone als Polygon	Abschnitt 5 und Aufnahme der Bearbeitung
7	Austritts-Alarm	Aufnahme und Tabelle aus Abschnitt 6
8	Strecke und Orte	Abschnitt 7
9	SMS-Alarme	Nachrichten aus Abschnitt 8
10	Ohne Hardware testen	Simulator und Szenario (Abschnitt 9)
11	Mit dem Patienten sprechen	Schema und Haken aus Abschnitt 10
12	Das simulierte Gerät	Aufnahme und Funktionen aus Abschnitt 11
13	Mit einem Befehl installieren	Abschnitt 12: die Befehle und die 4 Schritte
14	Stand der Arbeiten	Tabelle aus Abschnitt 14
15	Nächste Schritte	Abschnitt 17

Anhang: Inhalt des Ordners und Start

Datei	Aufgabe
`setup.js, start.js, install.ps1, install.sh, Starten.bat, starten.sh`	Konfigurationsassistent, Starter und Installer mit einem Befehl
`ttn/uplink_decoder.js`	Dekodierung der Position (8 Byte) und der Lesebestätigung
`ttn/downlink_encoder.js`	Kodierung des LED-Befehls, des Textes und des Sprachsignals
`web/index.html, style.css, app.js`	Oberfläche, Stile und Logik der Seite
`web/config.example.js` und `web/config.js`	Vorlage der Parameter (veröffentlicht) und echte Parameter mit den Schlüsseln, vom Assistenten angelegt (vom Repository ausgeschlossen)
`tools/simulate.html`, `tools/serve.js`	Positions-Simulator; lokaler Webserver ohne Abhängigkeiten
`tools/sensor.html`	Seite des simulierten Geräts: Karte, Position, Mikrofon, Nachrichten
`tools/voice-relay.js`, `tools/voice-tts.js`	Lokales Sprach-Relay (Nachrichten, Audio, Benachrichtigungen) und Sprachsynthese des Textes
`tools/test-voice.js`	74 automatische Tests des Relays (`node tools/test-voice.js`)
`tools/sms-proxy.js`	Lokaler Proxy für den SMS-Versand
`tools/sms-proxy.env.example`	Vorlage der Proxy-Konfiguration (die echte Datei ist vom Repository ausgeschlossen); in der Seite eingegebene Nummern liegen in `tools/sms-recipients.json` , ebenfalls ausgeschlossen
`firmware/heltec_gps_geofence/`	Code des Heltec-Geräts und seine Anleitung
`docs/`	Konfiguration der Plattform, Testanleitung, technische Erklärung, SMS-Anleitung, Nachrichtenanleitung

Das Projekt starten

Doppelklick auf `Starten.bat` (Windows) oder in einem Terminal, im Projektordner:

```
node start.js # Webserver + SMS-Alarme + Öffnen des Browsers: http://localhost:8080/web/
```

Dokument erstellt am 3. November 2026 aus dem tatsächlichen Stand des Projekts und der durchgeführten Versuche; deutsche Fassung der französischen Zusammenfassung. Die Aufnahmen stammen aus der deutschen Oberfläche im Demo-Modus, mit einer Beispielunterhaltung in einem getrennten Datenordner; für ihre Erstellung wurde weder eine SMS noch eine Nachricht gesendet.